Comprehensive SQL Concepts Handbook

Introduction to SQL

SQL (Structured Query Language) is the standard language used to manage relational databases. A database stores data in tables consisting of rows and columns. SQL allows users to create, read, update, and delete data. SQL is used in MySQL, PostgreSQL, Oracle, SQL Server, SQLite, and many other database systems.

Database Fundamentals

Key concepts include Database, Table, Record (Row), Field (Column), Schema, Constraints, Keys, Relationships, and Data Integrity. Relational databases organize data into related tables.

Keys in SQL

Primary Key uniquely identifies each row. Foreign Key creates relationships between tables. Candidate Key can uniquely identify records. Composite Key uses multiple columns. Unique Key prevents duplicate values.

SQL Data Types

Numeric Types: INT, BIGINT, FLOAT, DECIMAL. String Types: CHAR, VARCHAR, TEXT. Date Types: DATE, TIME, DATETIME, TIMESTAMP. Boolean Types and NULL values. Choosing proper data types improves performance and storage efficiency.

DDL Commands

CREATE: Creates tables and databases. ALTER: Modifies existing structures. DROP: Permanently removes objects. TRUNCATE: Deletes all rows quickly. RENAME: Renames objects.

DML Commands

INSERT adds records. UPDATE modifies records. DELETE removes records. These commands manipulate data stored in tables.

DQL Commands

SELECT retrieves data from tables. WHERE filters records. DISTINCT removes duplicates. ORDER BY sorts results. LIMIT restricts output size.

Operators

Arithmetic Operators: +, -, *, /. Comparison Operators: =, !=, <, >, <=, >=. Logical Operators: AND, OR, NOT. Special Operators: BETWEEN, IN, LIKE, EXISTS.

Aggregate Functions

COUNT(), SUM(), AVG(), MIN(), MAX(). Used with GROUP BY to summarize data.

Joins

INNER JOIN returns matching rows. LEFT JOIN returns all rows from left table. RIGHT JOIN returns all rows from right table. FULL JOIN returns all matching and non-matching rows. SELF JOIN joins a table with itself. CROSS JOIN produces Cartesian product.

Subqueries

A query nested inside another query. Can be used in SELECT, FROM, WHERE, and HAVING clauses. Correlated subqueries execute once per outer-row.

Views

Views are virtual tables created from SQL queries. They simplify complex queries and improve security.

Indexes

Indexes speed up searching and retrieval. Clustered and Non-clustered indexes improve performance. Too many indexes can slow INSERT and UPDATE operations.

Constraints

NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT constraints enforce data integrity.

Normalization

1NF removes repeating groups. 2NF removes partial dependency. 3NF removes transitive dependency. BCNF improves database design further. Normalization reduces redundancy.

Transactions and ACID

Atomicity: Complete or rollback. Consistency: Maintains valid state. Isolation: Transactions do not interfere. Durability: Committed changes persist. Commands: COMMIT, ROLLBACK, SAVEPOINT.

Stored Procedures

Reusable SQL programs stored in database. Improve code reuse and performance.

Functions

Built-in functions include string, date, numeric, conversion, and aggregate functions. User-defined functions extend functionality.

Triggers

Automatically execute on INSERT, UPDATE, or DELETE events. Used for auditing, validation, and automation.

Window Functions

ROW_NUMBER(), RANK(), DENSE_RANK(), LEAD(), LAG(), NTILE(). Used for advanced analytical queries.

Common Table Expressions (CTE)

CTEs improve readability of complex queries. Recursive CTEs are useful for hierarchical data.

SQL Security

GRANT and REVOKE manage permissions. Roles simplify privilege management. Database security protects sensitive data.

Query Optimization

Use indexes wisely. Avoid SELECT * in production. Analyze execution plans. Optimize joins and filtering conditions.

MySQL vs PostgreSQL vs SQL Server

MySQL is popular for web applications. PostgreSQL offers advanced standards compliance. SQL Server integrates well with Microsoft ecosystems.

SQL Interview Preparation

Topics frequently asked: Joins, Normalization, Indexes, ACID Properties, Primary Key vs Unique Key, DELETE vs TRUNCATE vs DROP, Subqueries, Window Functions, and Query Optimization.

Important SQL Examples

```
CREATE DATABASE company;

CREATE TABLE Employees (
id INT PRIMARY KEY,
name VARCHAR(100),
salary DECIMAL(10,2),
department VARCHAR(50)
);

INSERT INTO Employees VALUES (1,'Viraj',50000,'IT');

SELECT * FROM Employees;

SELECT department, AVG(salary)
FROM Employees
GROUP BY department;

SELECT *
FROM Employees
WHERE salary > 40000;

UPDATE Employees
SET salary = 55000
WHERE id = 1;

DELETE FROM Employees
WHERE id = 1;

SELECT e.name, d.department_name
FROM Employees e
INNER JOIN Departments d
ON e.department = d.department_id;
```